

Planowanie z wykorzystaniem języka PDDL

Mikołaj Kamiński 280596

1. Wstęp

Celem listy jest zapoznanie się z językiem **PDDL** służącym do opisu problemów planowania. PDDL pozwala na deklaratywne definiowanie:

- **Domeny** — zbioru typów, predykatów i akcji opisujących możliwe operacje,
- **Problemu** — konkretnej instancji z obiektami, stanem początkowym i celem.

Planer automatycznie generuje sekwencję akcji (plan) prowadzącą od stanu początkowego do stanu spełniającego cel. W ramach listy rozwiązano trzy zadania o rosnącej złożoności, a następnie przeprowadzono eksperymenty porównawcze z różnymi algorytmami przeszukiwania i wariantami topologii.

Wykorzystane solvery: - **BFWS** (Best-First Width Search) — online solver na editor.planning.domains, - **pyperplan** — solver PDDL w Pythonie z możliwością wyboru algorytmu przeszukiwania i heurystyki.

2. Zadanie 1 — Transport paczek (25 pkt)

2.1 Opis problemu

Zaprojektowano model logistyczny transportu paczek między polskimi miastami z wykorzystaniem trzech rodzajów transportu: **drogowego** (ciężarówki), **lotniczego** (samoloty) i **wodnego** (statki). Każdy środek transportu wymaga odpowiedniej infrastruktury — drogi między miastami, lotniska na obu końcach trasy lotniczej, porty na końcach trasy morskiej.

2.2 Model domeny — domain.pddl

Zdefiniowałem 5 typów obiektów i 12 predykatów:

```
(define (domain transport-logistics)
  (:requirements :strips :typing)
  (:types city package truck plane ship)

  (:predicates
    (at-truck ?t - truck ?c - city)
    (at-plane ?p - plane ?c - city)
    (at-ship ?s - ship ?c - city)
    (at-package ?p - package ?c - city)
    (in-truck ?p - package ?t - truck)
    (in-plane ?p - package ?pl - plane)
    (in-ship ?p - package ?s - ship))
```

```

(road ?c1 - city ?c2 - city)
(air-route ?c1 - city ?c2 - city)
(sea-route ?c1 - city ?c2 - city)
(has-airport ?c - city)
(has-port ?c - city)
)

;; === TRANSPORT DROGOWY ===
(:action drive
 :parameters (?t - truck ?from - city ?to - city)
 :precondition (and (at-truck ?t ?from) (road ?from ?to))
 :effect (and (at-truck ?t ?to) (not (at-truck ?t ?from)))
)

;; === TRANSPORT LOTNICZY ===
(:action fly
 :parameters (?pl - plane ?from - city ?to - city)
 :precondition (and
  (at-plane ?pl ?from)
  (has-airport ?from)
  (has-airport ?to)
  (air-route ?from ?to)
 )
 :effect (and (at-plane ?pl ?to) (not (at-plane ?pl ?from)))
)

;; === TRANSPORT WODNY ===
(:action sail
 :parameters (?s - ship ?from - city ?to - city)
 :precondition (and
  (at-ship ?s ?from)
  (has-port ?from)
  (has-port ?to)
  (sea-route ?from ?to)
 )
 :effect (and (at-ship ?s ?to) (not (at-ship ?s ?from)))
)

;; === ZAŁADUNEK / ROZŁADUNEK - CIĘŻARÓWKA ===
(:action load-truck
 :parameters (?p - package ?t - truck ?c - city)
 :precondition (and (at-package ?p ?c) (at-truck ?t ?c))
 :effect (and (in-truck ?p ?t) (not (at-package ?p ?c)))
)

(:action unload-truck

```

```

:parameters (?p - package ?t - truck ?c - city)
:precondition (and (in-truck ?p ?t) (at-truck ?t ?c))
:effect (and (at-package ?p ?c) (not (in-truck ?p ?t)))
)

;; === ZAŁADUNEK / ROZŁADUNEK - SAMOLOT ===
(:action load-plane
:parameters (?p - package ?pl - plane ?c - city)
:precondition (and (at-package ?p ?c) (at-plane ?pl ?c))
:effect (and (in-plane ?p ?pl) (not (at-package ?p ?c)))
)

(:action unload-plane
:parameters (?p - package ?pl - plane ?c - city)
:precondition (and (in-plane ?p ?pl) (at-plane ?pl ?c))
:effect (and (at-package ?p ?c) (not (in-plane ?p ?pl)))
)

;; === ZAŁADUNEK / ROZŁADUNEK - STATEK ===
(:action load-ship
:parameters (?p - package ?s - ship ?c - city)
:precondition (and (at-package ?p ?c) (at-ship ?s ?c))
:effect (and (in-ship ?p ?s) (not (at-package ?p ?c)))
)

(:action unload-ship
:parameters (?p - package ?s - ship ?c - city)
:precondition (and (in-ship ?p ?s) (at-ship ?s ?c))
:effect (and (at-package ?p ?c) (not (in-ship ?p ?s)))
)
)

```

Zdefiniowałem **9 akcji**:

Grupa	Akcje	Warunki wstępne
Transport drogowy	drive	Ciężarówka w mieście źródłowym + istnieje droga
Transport lotniczy	fly	Samolot w mieście + lotniska na obu końcach + trasa lotnicza
Transport wodny	sail	Statek w mieście + porty na obu końcach + trasa morska

Grupa	Akcje	Warunki wstępne
Załadunek/rozładunek	load-truck, unload-truck, load-plane, unload-plane, load-ship, unload-ship	Paczka i pojazd w tym samym mieście

2.3 Definicja problemu — problem.pddl

```
(define (problem deliver-packages)
  (:domain transport-logistics)

  (:objects
    wroclaw poznan gdansk krakow warszawa - city
    paczka1 paczka2 paczka3 paczka4 - package
    truck1 truck2 - truck
    plane1 - plane
    ship1 - ship
  )

  (:init
    ;; --- Pozycje pojazdów ---
    (at-truck truck1 wroclaw)
    (at-truck truck2 warszawa)
    (at-plane plane1 wroclaw)
    (at-ship ship1 gdansk)

    ;; --- Pozycje paczek ---
    (at-package paczka1 wroclaw) ; cel: gdansk
    (at-package paczka2 krakow) ; cel: poznan
    (at-package paczka3 warszawa) ; cel: wroclaw
    (at-package paczka4 gdansk) ; cel: krakow

    ;; --- Drogi (dwukierunkowe) ---
    (road wroclaw poznan) (road poznan wroclaw)
    (road poznan gdansk) (road gdansk poznan)
    (road wroclaw krakow) (road krakow wroclaw)
    (road krakow warszawa) (road warszawa krakow)
    (road poznan warszawa) (road warszawa poznan)

    ;; --- Trasy lotnicze (dwukierunkowe) ---
    (air-route wroclaw warszawa) (air-route warszawa wroclaw)
    (air-route wroclaw gdansk) (air-route gdansk wroclaw)
```

```

;; --- Trasy morskie (dwukierunkowe) ---
(sea-route gdansk warszawa) (sea-route warszawa gdansk)

;; --- Infrastruktura ---
(has-airport wroclaw)
(has-airport warszawa)
(has-airport gdansk)
(has-port gdansk)
(has-port warszawa)
)

(:goal
  (and
    (at-package paczka1 gdansk)
    (at-package paczka2 poznan)
    (at-package paczka3 wroclaw)
    (at-package paczka4 krakow)
  )
)
)

```

Sieć transportowa obejmuje 5 miast i wygląda następująco:

```

          Gdańsk
          / |
        morze |
          / | droga
Warszawa |
 | \ |
droga droga |
 | \ |
Kraków Poznań
 \ /
droga droga
 \ /
Wrocław ---- (lot) ----> Warszawa
 \---- (lot) ----> Gdańsk

```

Obiekty: - 5 miast: Wrocław, Poznań, Gdańsk, Kraków, Warszawa - **4 paczki z przypisanymi trasami:** - paczka1: startuje z **Wrocławia** cel: **Gdańsk** - paczka2: startuje z **Krakowa** cel: **Poznań** - paczka3: startuje z **Warszawy** cel: **Wrocław** - paczka4: startuje z **Gdańska** cel: **Kraków** - 2 ciężarówki (Wrocław, Warszawa), 1 samolot (Wrocław), 1 statek (Gdańsk)

Infrastruktura: Lotniska w: Wrocław, Warszawa, Gdańsk. Porty w: Gdańsk, Warszawa.

2.4 Wygenerowany plan (BFWS, bez kosztów)

Solver BFWS wygenerował plan o długości **16 kroków**:

```
(load-truck paczka3 truck2 warszawa)
(drive truck1 wroclaw krakow)
(drive truck2 warszawa krakow)
(load-truck paczka2 truck2 krakow)
(drive truck2 krakow wroclaw)
(load-truck paczka1 truck2 wroclaw)
(unload-truck paczka3 truck2 wroclaw)
(drive truck2 wroclaw poznan)
(unload-truck paczka2 truck2 poznan)
(drive truck2 poznan gdansk)
(load-truck paczka4 truck2 gdansk)
(unload-truck paczka1 truck2 gdansk)
(drive truck2 gdansk poznan)
(drive truck2 poznan warszawa)
(drive truck2 warszawa krakow)
(unload-truck paczka4 truck2 krakow)
```

Obserwacja: Plan korzysta **wyłącznie z ciężarówek** — samolot i statek nie zostały użyte. Jest to efekt optymalizacji długości planu: transport lotniczy wymaga dodatkowych akcji załadunku/rozładunku (load-plane + fly + unload-plane = 3 akcje), podczas gdy bezpośredni przejazd ciężarówką to 1 akcja. Planer wybiera krótszy plan w sensie liczby akcji.

2.5 Wersja z kosztami — domain_costs.pddl

Aby uwzględnić realistyczne koszty transportu, rozszerzono domenę o :action-costs:

```
(:requirements :strips :typing :action-costs)
(:functions (total-cost) - number)
```

Przypisane koszty:

Akcja	Koszt	Uzasadnienie
drive	10	Transport drogowy — średni koszt
fly	50	Transport lotniczy — drogi ale szybki
sail	5	Transport morski — najtańszy
load/unload-truck	1	Przeładunek ciężarówki
load/unload-plane	2	Przeładunek samolotu
load/unload-ship	1	Przeładunek statku

Wygenerowany plan (BFWS, z kosztami) — 20 kroków, koszt = 20:

```

(load-plane paczka1 plane1 wroclaw)
(fly plane1 wroclaw gdansk)
(unload-plane paczka1 plane1 gdansk)
(drive truck1 wroclaw krakow)
(load-truck paczka2 truck1 krakow)
(drive truck1 krakow warszawa)
(drive truck1 warszawa poznan)
(unload-truck paczka2 truck1 poznan)
(drive truck1 poznan warszawa)
(load-truck paczka3 truck1 warszawa)
(drive truck2 warszawa poznan)
(drive truck2 poznan gdansk)
(drive truck1 warszawa poznan)
(drive truck1 poznan wroclaw)
(unload-truck paczka3 truck1 wroclaw)
(load-truck paczka4 truck2 gdansk)
(drive truck2 gdansk poznan)
(drive truck2 poznan wroclaw)
(drive truck2 wroclaw krakow)
(unload-truck paczka4 truck2 krakow)

```

Wnioski: W wersji z kosztami planer **wykorzystał samolot** do transportu paczka1 do Gdańska (fly, koszt 50+2+2=54), ale plan jest dłuższy (20 vs 16). Solver minimalizuje łączny koszt, nie długość planu. ### 2.6 Eksperyment A: Porównanie algorytmów przeszukiwania

Przeprowadzono porównanie 9 konfiguracji algorytmów przeszukiwania dostępnych w solverze **pyperplan** na tym samym problemie (pełna topologia, bez kosztów):

Algorytm	Heurystyka	Długość planu	Węzły ekspandowane	Czas [s]	Optymalny?
BFS	—	15	739 884	31.0	
A*	LmCut	15	1 137	22.0	
A*	hFF	15	354	0.8	
A*	hAdd	16	42	0.094	
GBF	hFF	18	40	0.078	
GBF	hAdd	18	25	0.031	
WA*	hFF	18	43	0.094	
EHS	hFF	19	70	0.14	
IDS	—	Timeout	Timeout	>60	—
BFWS (online)	—	16	206	0.001	

Niestety **pyperplan** nie dał rady wygenerować planu dla wariantu z kosztami, jako że solver ten obsługuje jedynie podstawowy standard PDDL (STRIPS) i

nie wspiera `:action-costs` ani `numeric-fluents`. Z tego powodu w Eksperymentcie A analizowano wariant bez kosztów.

Wnioski z eksperymentu:

1. **Trade-off jakość vs szybkość:** Algorytmy takie jak BFS i $A^*(LmCut)$ znajdują gwarantowanie optymalny plan (15 kroków), ale przeszukują ogromne przestrzenie (739 884 i 1 137 węzłów). Z kolei GBF(hAdd) znajduje plan 18-krokowy w zaledwie 25 węzłach — wykonując się w ułamek sekundy (1000x szybciej niż BFS).
2. **Wpływ heurystyki:** Heurystyki niegwarantujące optymalności (np. hFF) mogą czasem doprowadzić do optymalnego planu ($A^* + hFF$ odnalazł plan długości 15 w 0.8s), jednak bywają niestabilne (hAdd dający długość 16). Osiągają za to bezkompromisową szybkość w porównaniu do restrykcyjnego LmCut.
3. **Algorytmy zachłanne (GBF)** są najszybsze z całej rodziny (zaledwie kilkadziesiąt milisekund), płacąc za to suboptymalnością (plany 18-krokowe).
4. **IDS** nie zdołał znaleźć rozwiązania w limicie 60 sekund. To pokazuje, jak olbrzymie ograniczenia ma niekierowane przeszukiwanie iteracyjne w tak dużych przestrzeniach stanów.
5. **BFWS** (16 kroków, wynik z internetowego solvera) błyskawicznie (0.001s) znajduje przyzwoity kompromis przy niewielkiej liczbie rozszerzonych węzłów (206), balansując pomiędzy optymalnością a wydajnością obliczeniową.

3. Zadanie 2 — Robot odkurzacz (15 pkt)

3.1 Opis problemu

Robot ma odwiedzić wszystkie pokoje i je odkurzyć. Problem modeluje klasyczny scenariusz robota sprząającego z trzema pokojami.

3.2 Model domeny i problem

`domain.pddl` — dwa typy (`robot`, `room`), trzy predykaty, dwie akcje:

```
(define (domain vacuum-robot)
  (:requirements :strips :typing)
  (:types robot room)
  (:predicates
    (at ?r - robot ?p - room)
    (dirty ?p - room)
    (clean ?p - room)
  )
  (:action move
```



```

:parameters (?r - robot ?from - room ?to - room)
:precondition (at ?r ?from)
:effect (and (not (at ?r ?from)) (at ?r ?to))
)
(:action vacuum
:parameters (?r - robot ?p - room)
:precondition (and (at ?r ?p) (dirty ?p))
:effect (and (clean ?p) (not (dirty ?p))))
)
)

```

problem.pddl — robot startuje w pokoj1, wszystkie pokoje brudne:

```

(define (problem clean-all-rooms)
  (:domain vacuum-robot)
  (:objects
    robot - robot
    pokoj1 pokoj2 pokoj3 - room
  )
  (:init
    (at robot pokoj1)
    (dirty pokoj1)
    (dirty pokoj2)
    (dirty pokoj3)
  )
  (:goal
    (and (clean pokoj1) (clean pokoj2) (clean pokoj3))
  )
)

```

3.3 Wygenerowany plan

Plan optymalny — **5 kroków** (A*(LmCut): 6 węzłów, czas ~0s):

```

(vacuum robot pokoj1)
(move robot pokoj1 pokoj3)
(vacuum robot pokoj3)
(move robot pokoj3 pokoj2)
(vacuum robot pokoj2)

```

3.4 Analiza i wnioski

- Plan jest **optymalny** — minimum to 3 akcje sprzątania + 2 przejścia = 5 kroków.
- Robot sprząta pokój, w którym się znajduje (pokoj1), a następnie odwiedza pozostałe, sprząając każdy.
- Kolejność pokoj1→pokoj3→pokoj2 (zamiast pokoj1→pokoj2→pokoj3) jest jedną z dwóch równoważnych optymalnych ścieżek — obie wymagają

dokładnie 5 kroków, ponieważ w modelu nie ma topologii połączeń (robot może się przenieść między dowolnymi pokojami).

4. Zadanie 3 — Robot z piłkami (10 pkt)

4.1 Opis problemu

Robot z dwoma ramionami (arm1, arm2) ma przenieść 4 piłki z room1 do room2. Robot może podnosić piłki, przemieszczać się między pokojami i odkładać piłki.

4.2 Analiza modelu

`domain.pddl` — trzy akcje:

```
(define (domain ball-moving-robot)
  (:requirements :strips :typing)
  (:types robot room ball arm)
  (:predicates
    (at ?r - robot ?rm - room)
    (inroom ?b - ball ?rm - room)
    (holding ?a - arm ?b - ball)
    (arm-empty ?a - arm)
  )
  (:action move
    :parameters (?r - robot ?from - room ?to - room)
    :precondition (at ?r ?from)
    :effect (and (not (at ?r ?from)) (at ?r ?to))
  )
  (:action pick-up
    :parameters (?r - robot ?a - arm ?b - ball ?rm - room)
    :precondition (and (at ?r ?rm) (inroom ?b ?rm) (arm-empty ?a))
    :effect (and (holding ?a ?b) (not (arm-empty ?a)) (not (inroom ?b ?rm)))
  )
  (:action put-down
    :parameters (?r - robot ?a - arm ?b - ball ?rm - room)
    :precondition (and (at ?r ?rm) (holding ?a ?b))
    :effect (and (inroom ?b ?rm) (arm-empty ?a) (not (holding ?a ?b)))
  )
)
```

`problem.pddl` — 4 piłki w room1, cel: wszystkie w room2:

```
(define (problem move-balls)
  (:domain ball-moving-robot)
  (:objects
    room1 room2 - room
    robot - robot
    ball1 ball2 ball3 ball4 - ball
  )
)
```

```

    arm1 arm2 - arm
  )
  (:init
    (at robot room1)
    (inroom ball1 room1)
    (inroom ball2 room1)
    (inroom ball3 room1)
    (inroom ball4 room1)
    (arm-empty arm1)
    (arm-empty arm2)
  )
  (:goal
    (and
      (inroom ball1 room2)
      (inroom ball2 room2)
      (inroom ball3 room2)
      (inroom ball4 room2)
    )
  )
)

```

4.3 Wygenerowany plan

Plan optymalny — **11 kroków** (A*(LmCut): 95 węzłów, czas 0.12s):

```

(pick-up robot arm2 ball3 room1)
(pick-up robot arm1 ball4 room1)
(move robot room1 room2)
(put-down robot arm2 ball3 room2)
(put-down robot arm1 ball4 room2)
(move robot room2 room1)
(pick-up robot arm2 ball1 room1)
(pick-up robot arm1 ball2 room1)
(move robot room1 room2)
(put-down robot arm2 ball1 room2)
(put-down robot arm1 ball2 room2)

```

4.4 Analiza optymalności — rola dwóch ramion

Plan dzieli się na **2 tury przenoszenia**, z których każda ma strukturę:

Tura: pick-up(arm1) → pick-up(arm2) → move → put-down(arm1) → put-down(arm2)

Każda tura = 5 akcji, plus 1 akcja move powrotna między turami = $2 \times 5 + 1$
= **11**.

5. Wnioski końcowe

Wybór algorytmu przeszukiwania ma zasadniczy wpływ na efektywność planowania. Algorytmy optymalne (BFS, A* z dopuszczalną heurystyką) gwarantują najkrótszy plan, ale kosztem eksponencjalnie większej przestrzeni przeszukiwania. Algorytmy zachłanne (GBF) są tysiące razy szybsze, akceptując suboptymalne plany. **Heurystyka LmCut** okazała się jedyną heurystyką gwarantującą optymalność w A*, redukując liczbę ekspandowanych węzłów z 740 292 (BFS) do 1 153 przy zachowaniu optymalnego planu.